Implement Client Script & UI Policy (Incident)

Create UI Policy on Incident

Description

Create a **UI Policy** on the Incident table to dynamically control the behavior of form fields based on specific conditions. Configure the UI Policy to make selected fields **mandatory, read-only, or visible** when the defined condition is met. This helps ensure that users provide the required information and follow consistent data-entry practices while creating or updating Incident records.

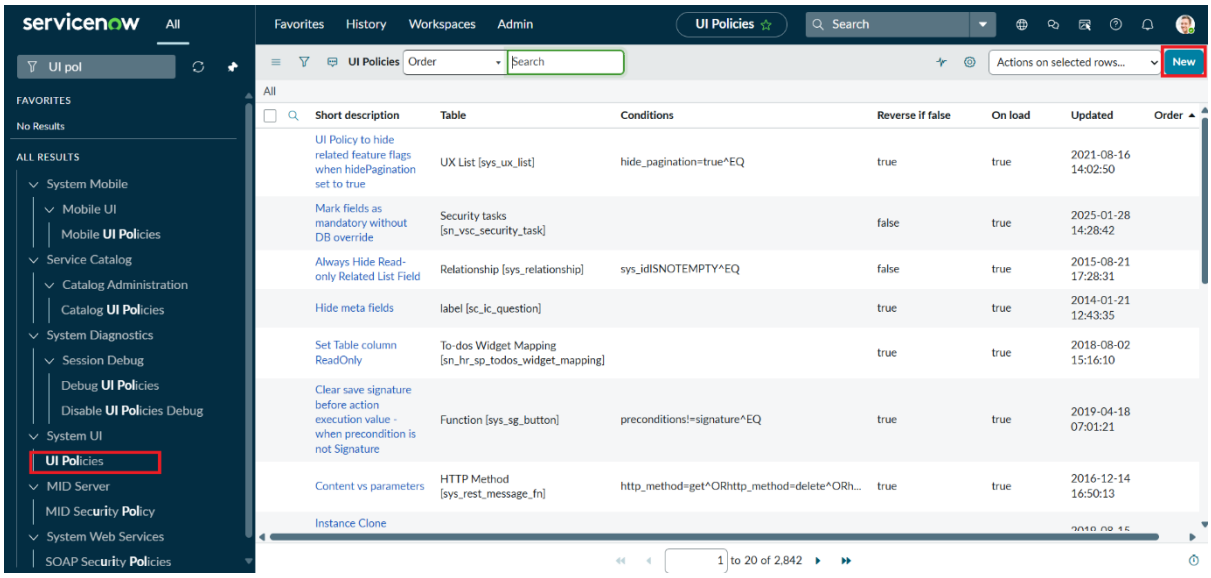
Create UI Policy

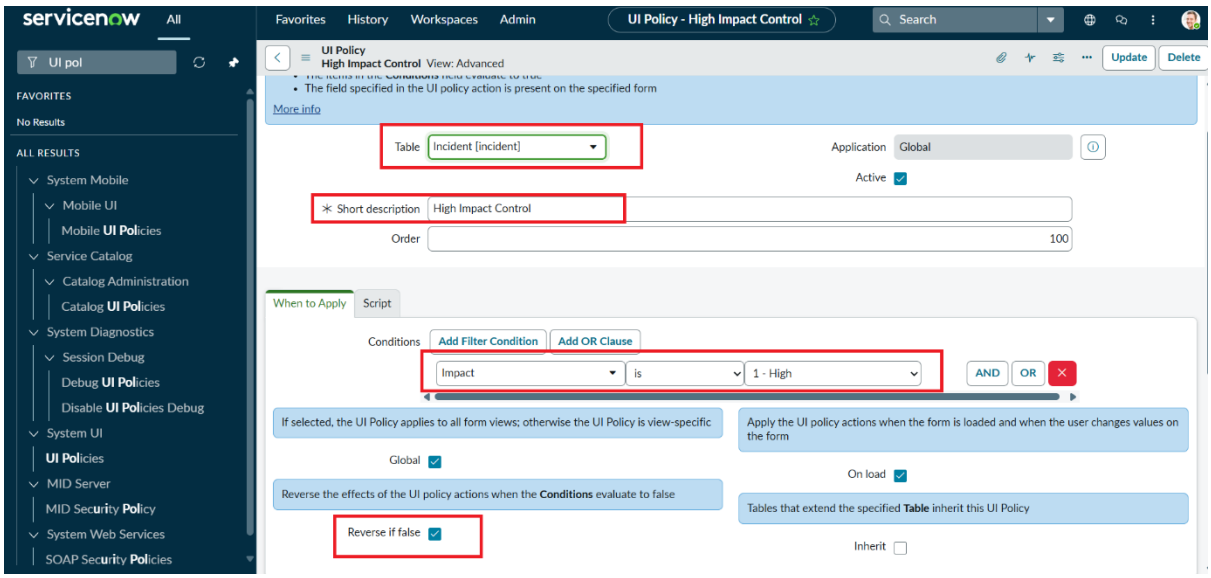
Description

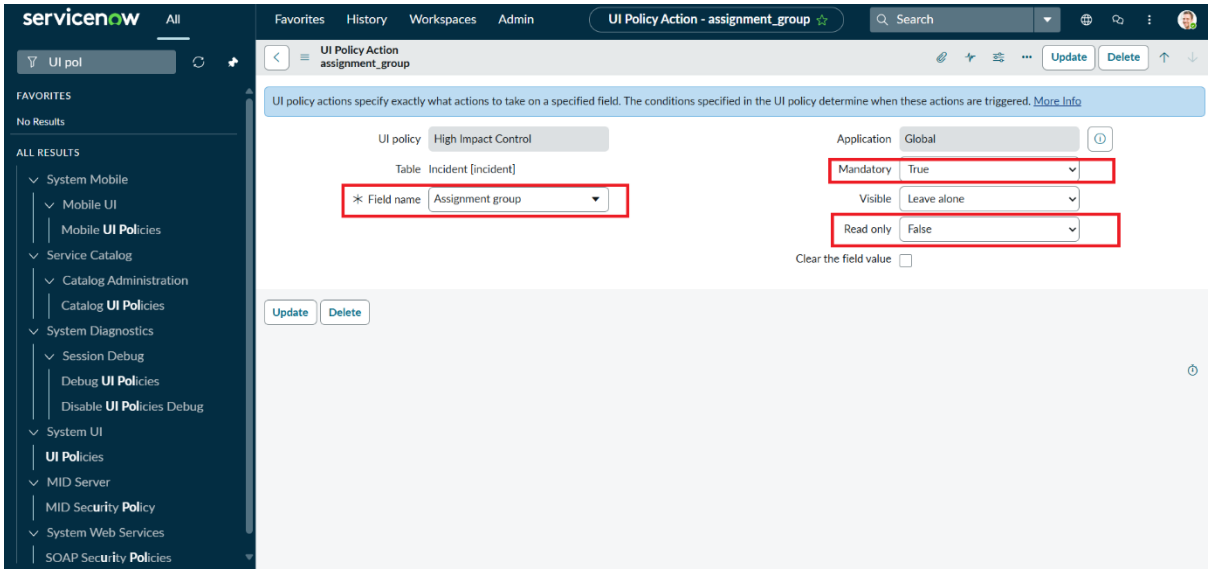
1. Log in to the ServiceNow instance with administrative or configuration access.
2. Navigate to:
   System UI → UI Policies
3. Click New to create a new UI Policy record.
4. Fill in the following details:
   - Name: High Impact Control
     This name clearly represents the purpose of the policy, which is to apply additional controls when an Incident has a high impact.
   - Table: Incident
     Specifies that the UI Policy applies to Incident records.
   - Active: true
     Enables the UI Policy so it is enforced on the Incident form.
5. Configure the Conditions section as follows:
   - Field: Impact
   - Operator: is

- Value: 1 – High

6. This condition ensures the UI Policy is triggered only when the Incident impact is set to High.
7. And Ui policy action Assignment group
8. Check mandatory
9. Set Reverse if false to true.
   This ensures that when the condition is not met, any field changes applied by the policy are automatically reverted.
10. Review the configuration to ensure accuracy.
11. Click Submit to save the UI Policy.







Create UI Policy Action – Urgency

Description

Create a UI Policy Action for the Urgency field on the Incident form to control its behavior based on the conditions defined in the UI Policy. Configure the action to make the Urgency field mandatory, visible, or read-only as required. This ensures that the appropriate urgency information is captured consistently and helps users prioritize incidents accurately.

Configure the Urgency Field

Description

Open the UI Policy Actions
Go to the High Impact Control UI Policy you created earlier. Scroll down to the UI Policy Actions related list. This is where you define the specific behavior of fields when the policy conditions are met.

Create a New Action
Click New to create a UI Policy Action. This lets you configure how individual fields behave when the UI Policy is triggered.

Fill in the Action Details

Field name: Urgency
Select the Urgency field on the Incident form. This is the field you want to control.

Read-only: true
Check this option to make the Urgency field read-only whenever the UI Policy condition is met (Impact = High). Users will be able to see the field but cannot change its value.

Visible: Leave this field as is (do not change).
By leaving it unchanged, the field will remain visible on the form, but its behavior will be controlled by the Read-only setting.

Submit the Action
Click Submit to save the UI Policy Action. The action is now linked to the High Impact Control UI Policy and will automatically apply to the Urgency field when an Incident has High Impact.

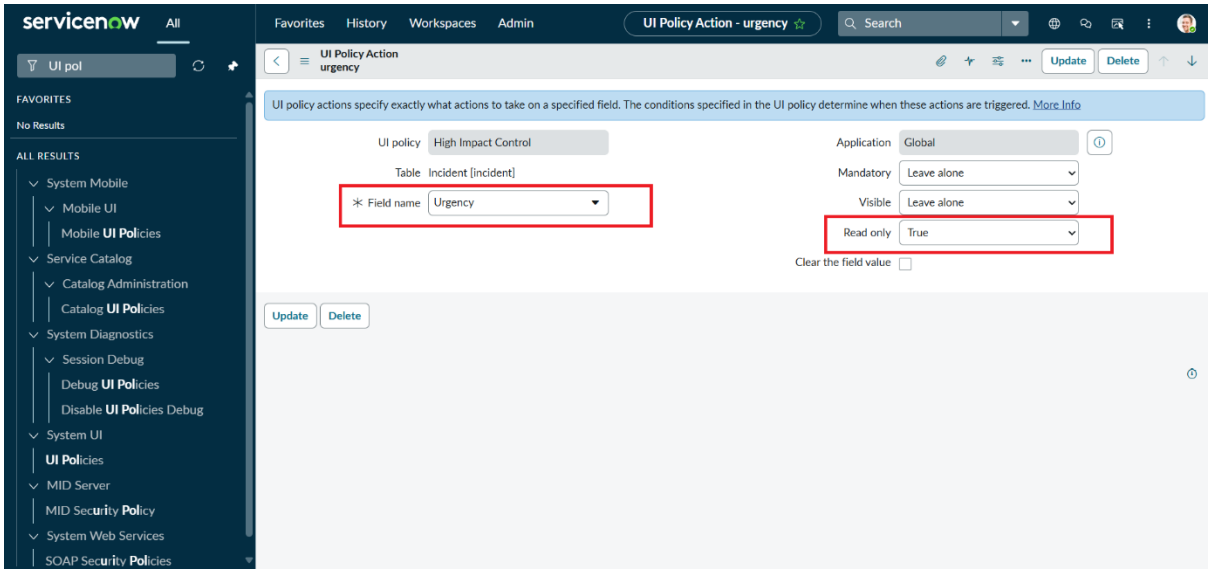
Optional Verification
To test the action:

Open an Incident record.

Set the Impact field to High.

Check that the Urgency field becomes read-only.

Change Impact to a different value to confirm that the field reverts back if needed.



Create onChange Client Script

Description

Create an onChange Client Script on the Incident form to dynamically perform an action whenever a specified field value is changed. The script should use the g_form API to update related fields or display appropriate messages based on the selected value. This helps automate form behavior, improve data accuracy, and provide users with a more interactive Incident form experience.

Create Client Script

Description

Navigate to Client Scripts:
Go to System UI → Client Scripts.

Click New to create a new Client Script.

Fill in the details:

Name: Auto set urgency for high impact – clearly shows the purpose.

Table: Incident – the script runs only on Incident records.

Type: onChange – executes when a field value changes.

Field name: Impact – triggers the script when Impact is updated.

Active: true – ensures the script runs immediately.

Paste the script in the script editor section.

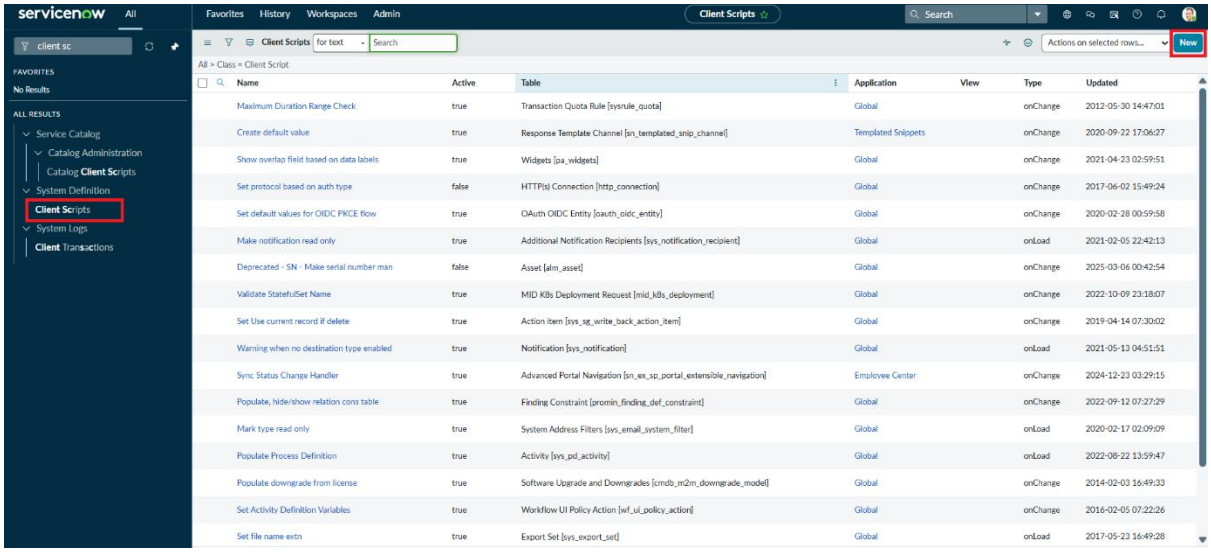
```
function onChange(control, oldValue, newValue, isLoading) {
   if (isLoading || newValue == '') {
      return;
   }

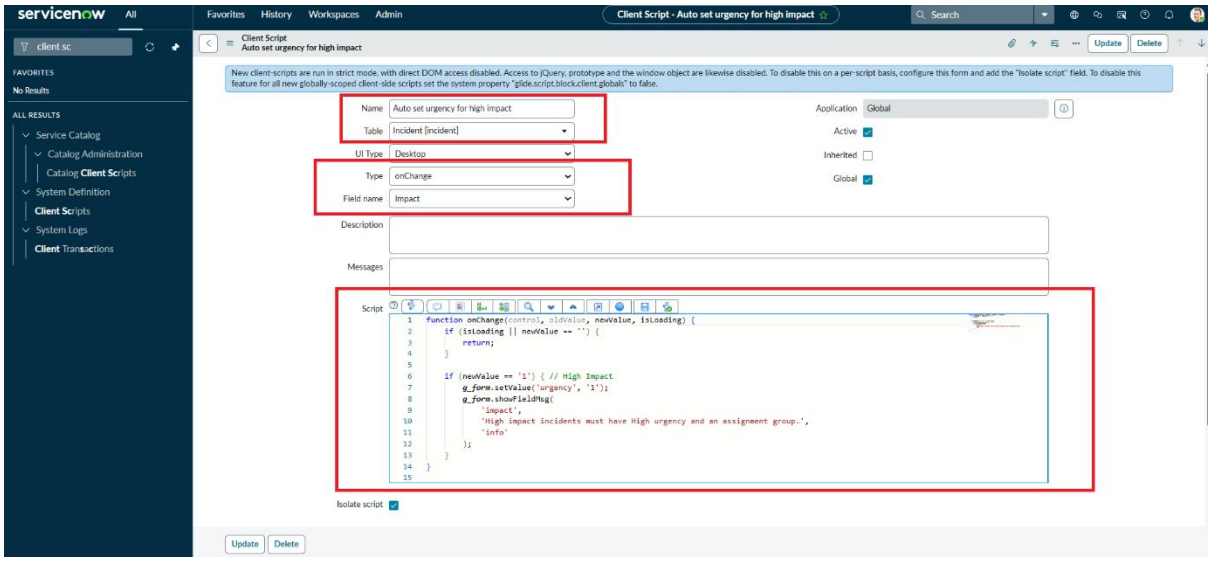
   if (newValue == '1') {
      g_form.setValue('urgency', '1');
      g_form.addInfoMessage('Urgency set to High for High impact incident.');
   }
}
```

Click Submit to save the Client Script.

Verify: Open an Incident record and change the Impact field to High. The Urgency field should update automatically

Click Submit.





Create onSubmit Client Script (Save Validation)

Description

Create an onSubmit Client Script on the Incident form to validate the information entered by the user before the record is saved or submitted. Use the g_form API to check whether the required conditions are satisfied and prevent submission when invalid or incomplete data is found. Display an appropriate message to guide the user in correcting the information before saving the Incident.

Create Client Script

Description

Navigate to Client Scripts:
Go to System UI → Client Scripts in your ServiceNow instance.

Click New to create a new Client Script.

Fill in the details:

Name: Prevent save if Assigned To missing – clearly indicates that the script prevents saving if the field is empty.

Table: Incident – applies the script only to Incident records.

Type: onSubmit – executes when the user tries to save the record.

Active: true – ensures the script is enforced immediately.

Paste the script into the script editor section. This script should check if the **Assigned To** field is empty and prevent the record from saving if it is.

```
function onSubmit() {
  if (g_form.getValue('impact') == '1' &&
    g_form.getValue('assigned_to') == '') {

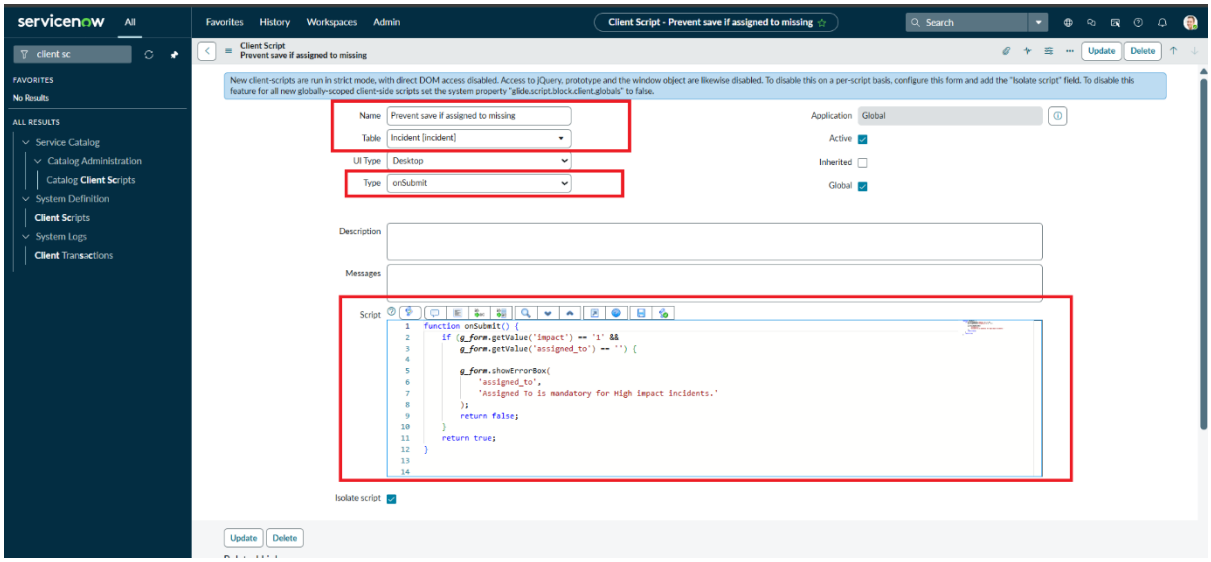
    g_form.showErrorBox(
      'assigned_to',
```

```
            'Assigned To is mandatory for High impact incidents.'
        );
        return false;
    }
    return true;
}
```

Click Submit to save the Client Script.

Verify: Try saving an Incident without filling the **Assigned To** field. The script should prevent the save and display a warning.

8. Click **Submit**.



Create onCellEdit Client Script

Description

Create an **onCellEdit Client Script** on the Incident table to control and validate field values when users edit Incident records directly from a **list view**. The script should check the updated value and allow or prevent the

change based on the defined conditions. This helps maintain data accuracy and ensures that users follow the required validation rules even when making inline edits from the Incident list.

Create Client Script

Description

Navigate to Client Scripts:
Go to System UI → Client Scripts in your ServiceNow instance.

Click New to create a new Client Script.

Fill in the details:

Name: Prevent state change via list edit – clearly indicates that the script prevents users from changing the State field directly in a list.

Table: Incident – applies the script only to Incident records.

Type: onCellEdit – runs whenever a field is edited directly in a list view.

Field name: State – the script will monitor changes to the State field.

Active: true – ensures the script is enforced immediately.

Paste the script into the script editor section. The script should prevent unauthorized changes to the State field when editing from the list view.

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {

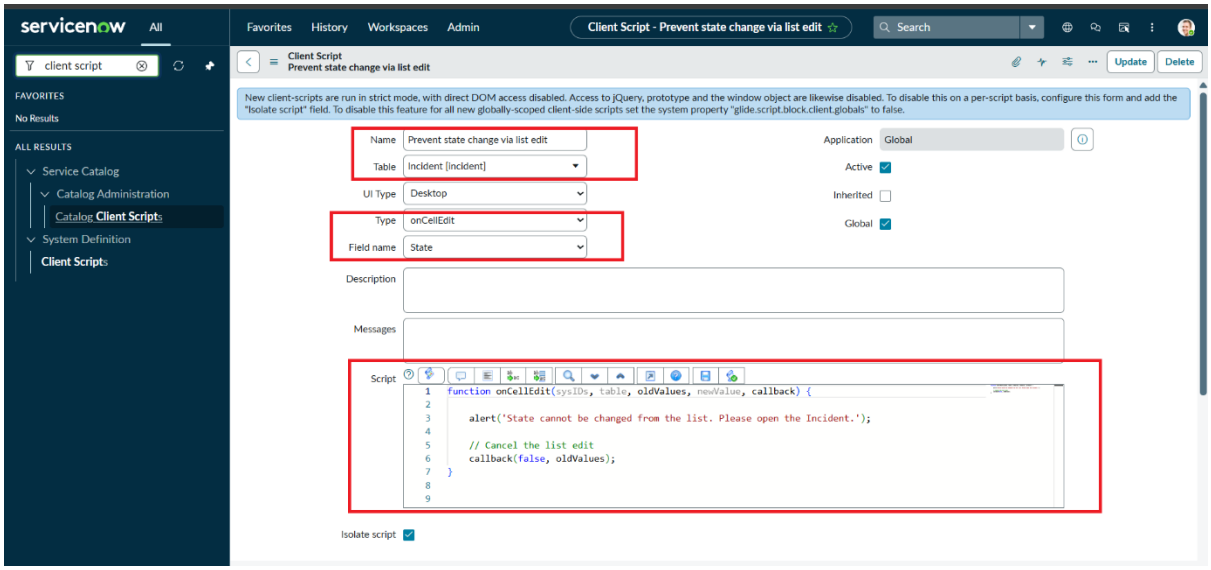
   alert('State cannot be updated using list editing. Please open the Incident.');
```

```
  callback(false);
}
```

Click Submit to save the Client Script.

Verify: Open an Incident list and try to change the State field directly in the list. The script should block the change as intended.

7. Click **Submit**.



Testing the Configuration

Description

Test all the configured **UI Policies, UI Policy Actions, and Client Scripts** on the Incident form and list view to ensure they work as expected. Verify that fields become **mandatory, visible, read-only, or updated** based on the defined conditions. Test different scenarios, including valid and invalid inputs, form submission, field changes, and inline list edits, and confirm that the expected validation messages and behaviors are triggered correctly.

Test Mandatory Enforcement

Description

Navigate to Incident:
Go to Incident → Create New to open a new Incident form.

Set the Impact Field:
Select High for the Impact field. This will trigger the UI Policy and Client Script behaviors you configured earlier.

Leave Assigned To Empty:
Do not fill in the Assigned To field. This will test the onSubmit Client Script that prevents saving if Assigned To is missing.

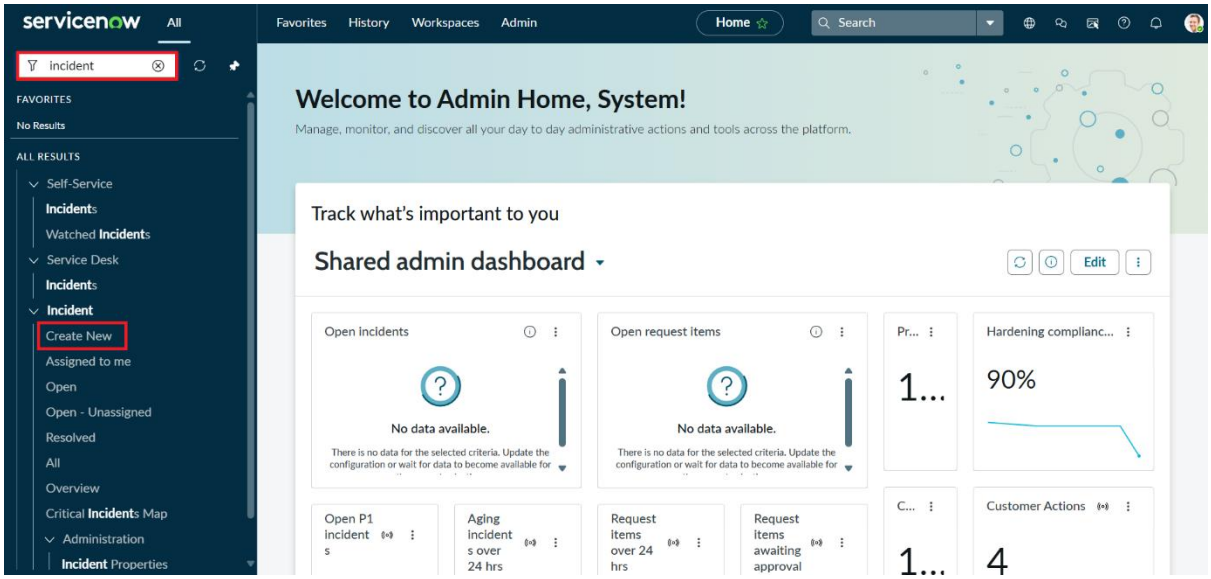
Click Submit:
Attempt to save the Incident record.

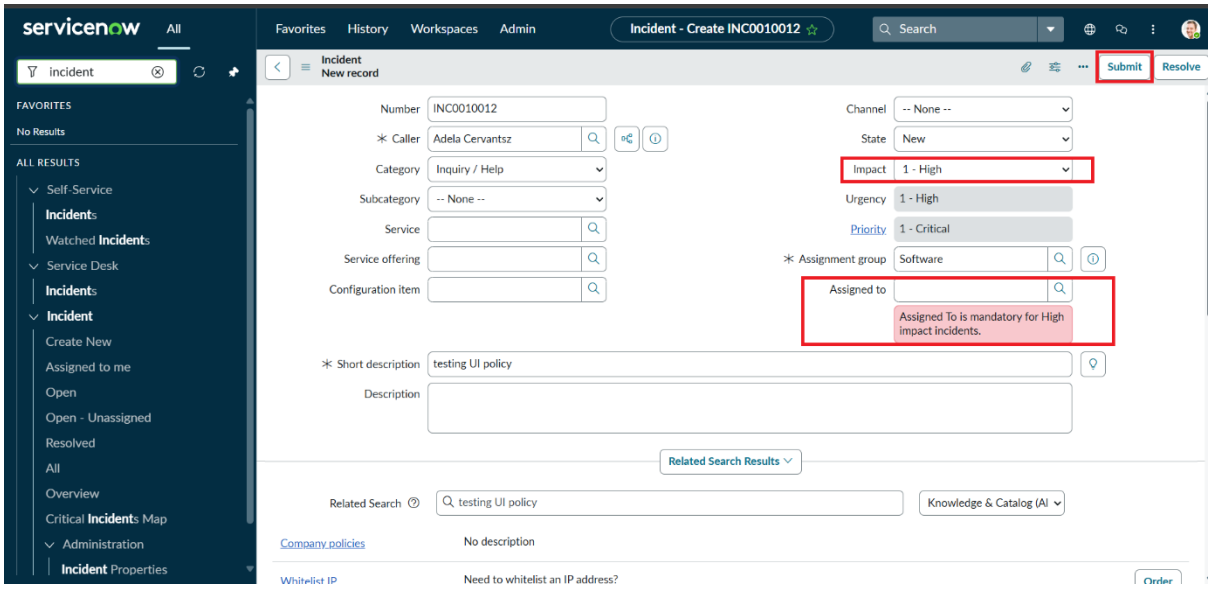
Verify the Result:

The Incident should not be saved because Assigned To is required when Impact is High.

An error message should appear, indicating that the Assigned To field cannot be empty.

Confirm that all UI Policy actions, such as read-only fields or automated urgency updates, are working as expected.





Test Successful Save

Description

Fill the Assigned To Field:
Open the Incident form and select a user in the Assigned To field. This ensures the onSubmit Client Script condition is satisfied.

Click Submit:
Attempt to save the Incident record.

Verify the Result:

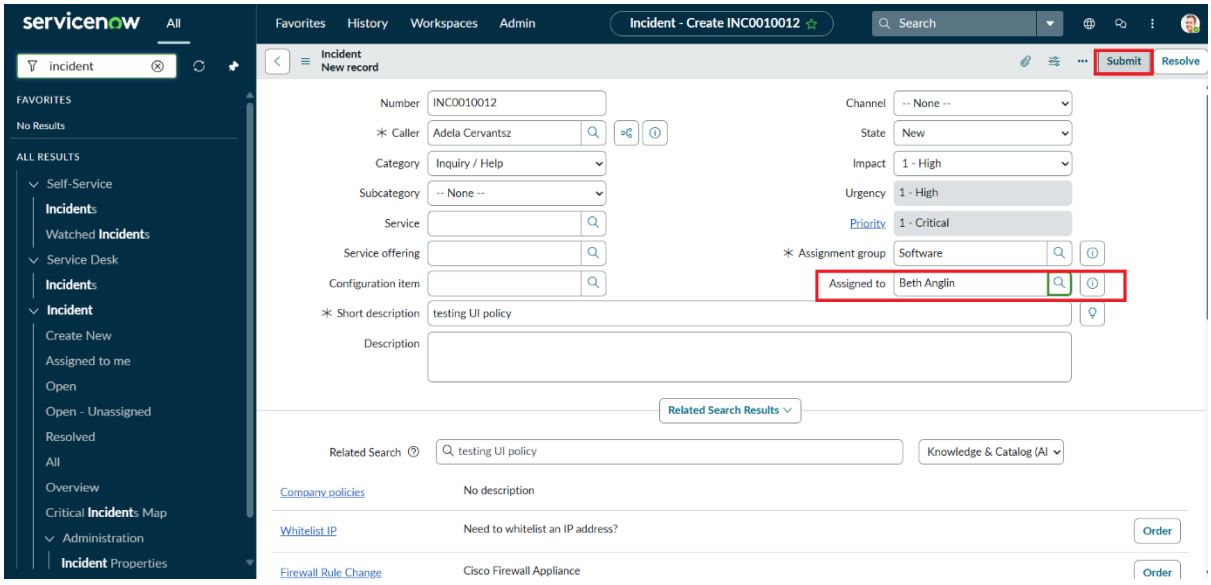
The Incident should be saved successfully without any errors.

Confirm that all other configured behaviors, such as:

Urgency auto-setting (from the onChange Client Script)

Read-only fields (from the UI Policy Action)

State restrictions (from the onCellEdit Client Script)
are functioning as expected



Reverse Condition Test

Description

Open an Existing Incident:
Go to Incident → Open and select an Incident where Impact is currently set to High.

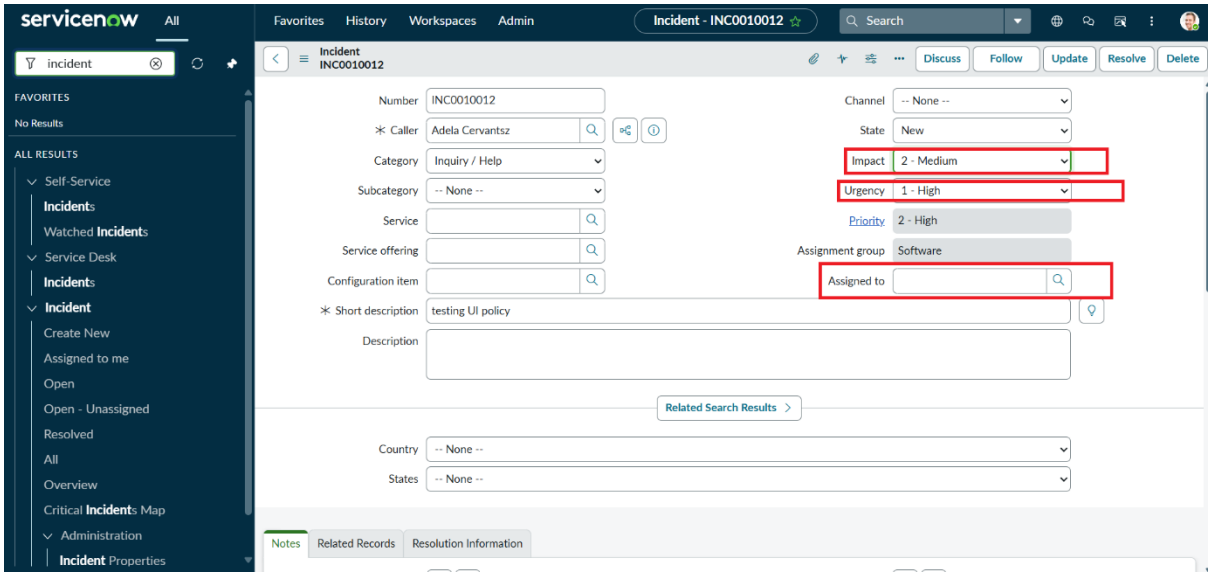
Change Impact:
Update the Impact field from High to Medium. This will test the Reverse if false functionality of your UI Policy and the behavior of your Client Scripts.

Verify the Field Behavior:

Assigned To should no longer be mandatory.

Urgency should now be editable, meaning the read-only UI Policy action is reversed.

Save the Record:
Click Submit to save the changes. The record should save successfully without errors, confirming that the reverse condition works as intended.



Test List Edit Blocking

Description

Navigate to the Incident List:
Go to Incident → All to view all Incident records in a list view.

Attempt a List Edit:

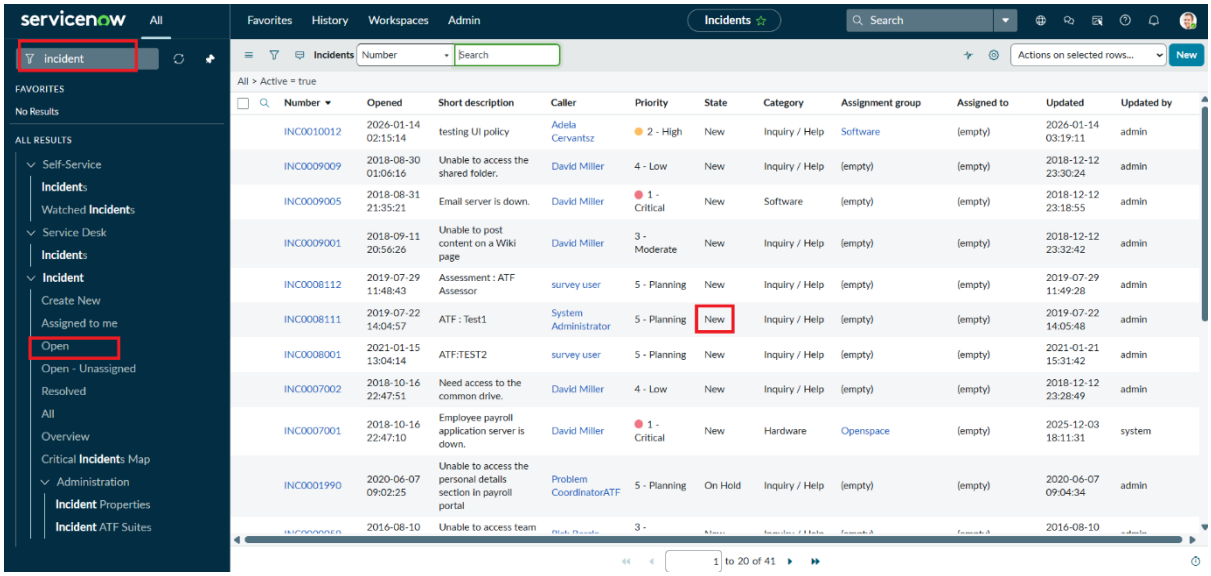
Locate any Incident in the list.

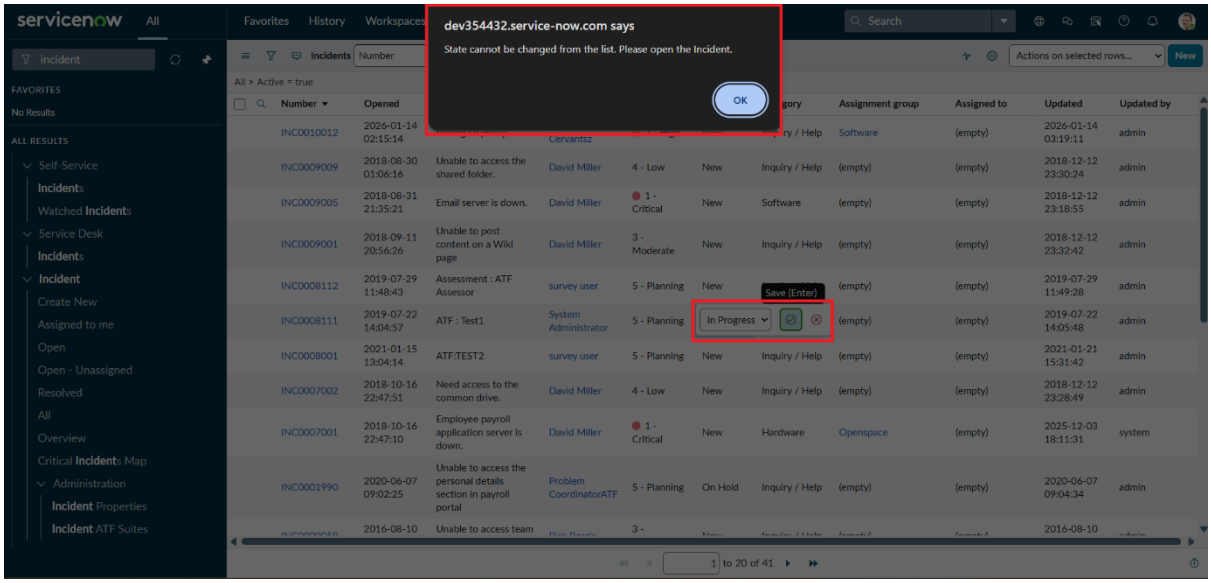
Double-click the State field to try editing it directly from the list.

Verify the Behavior:

An alert message should appear, indicating that direct changes to the State field are not allowed.

The State value should remain unchanged, confirming that the onCellEdit Client Script is working correctly.





Test Form-Based Update

Description

Open the Incident Record:
Navigate to the same Incident record you used in previous tests.

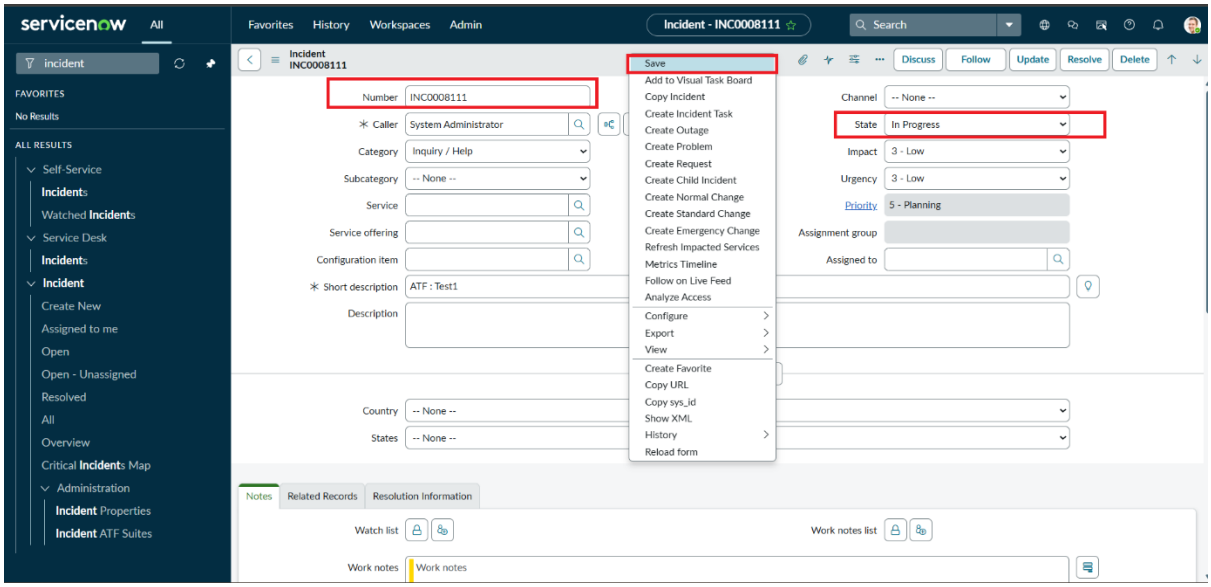
Change the State:
On the Incident form, update the State field to a new value. This tests that changes via the form (not the list) are allowed.

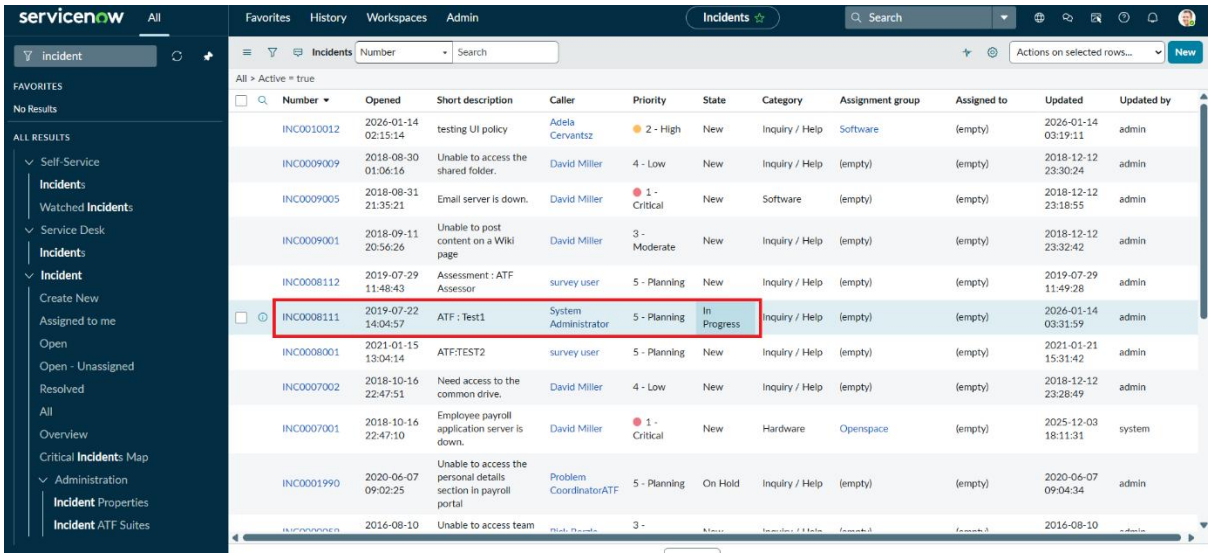
Click Update:
Save the record by clicking Update.

Verify the Result:

The State change should be saved successfully.

Confirm that other fields, such as Assigned To and Urgency, still follow the configured UI Policy and Client Script rules.





Conclusion

Description

This project successfully demonstrates the implementation of UI Policies, UI Policy Actions, and Client Scripts on the ServiceNow Incident table. The configuration enables dynamic field behavior, validation, and controlled data entry based on user actions and predefined conditions. By testing different scenarios, the Incident form and list view become more user-friendly, consistent, and reliable, helping improve overall data quality and incident management efficiency

Conclusion

Description

The Implement Client Script & UI Policy (Incident) micro project effectively demonstrates how UI Policies and Client Scripts can work together to enforce dynamic field behavior, automate updates, and prevent incorrect data submission on Incident forms. By using onChange, onSubmit, and onCellEdit scripts alongside UI Policy actions, the project ensures fields like Assigned To and Urgency respond appropriately to changes in Impact, maintaining clean and consistent data. This solution not only enforces mandatory fields and controls user interactions but also remains lightweight, efficient, and easy to implement, making it ideal for short-duration micro projects while showcasing practical best practices for improving form usability and data integrity in ServiceNow.